AI-Assisted Patch Antenna Design Using a Hybrid Encoder-Decoder Architecture

Wednesday, August 27, 2025

Abstract

The antenna is designed on an FR4 substrate (150 mm × 150 mm × 1.57 mm, dielectric constant 4.4) with a microstrip line feed and an inset configuration. A binary grid-based input representation of the antenna geometry is adopted, where a 10 × 10 matrix encodes the patch layout (0 = substrate, 1 = patch metallization). A hybrid encoder-decoder deep learning model is developed to map geometry to electromagnetic (EM) performance metrics, including return loss (S11) with respected Frequency, (VSWR, directivity, gain, and radiation efficiency is not going to be included in the initial design). The encoder employs a Graph Neural Network (GNN)-Transformer to effectively capture spatial relationships, while the decoder is based on the FEDformer architecture for accurate sequence-to-sequence regression. The model is trained on a dataset of 1000 full-wave simulations generated using HFSS.

1. Antenna Design Specification

- **Substrate Material:** FR4
- **Substrate Dimensions:** 150 mm × 150 mm × 1.57 mm
- **Patched Area Dimensions (variable area):** 100 mm × 100 mm
- **Dielectric Constant (εr):** 4.4
- **Antenna Type:** Microstrip patch antenna with microstrip line feed and inset
- **Geometry Encoding:** 10 × 10 binary array (0 = substrate, 1 = metallization patch, each binary represents 10 mm x 10 mm area)
- **Frequency Range**: 1.0 to 6.0 GHz with6 points
- **Outputs of AI Model:**
    - S11 (return loss)

2. Methodology

2.1. Geometry Encoding

The antenna patch is discretized into a 10 × 10 binary grid (total of 100). This encoding captures the layout of the metallization while excluding the feed line. The exclusion is justified because the feed structure is generally fixed across designs, while the patch geometry primarily governs radiation characteristics. By isolating the patch design, the model focuses on the radiating structure and avoids unnecessary complexity in the input representation.

3.2 Dataset Generation

A total of 1000 antenna configurations are simulated using a full-wave solver (HFSS). Each simulation provides frequency-dependent results for S11 (VSWR, gain, directivity, and efficiency are going to be ignored for the initial stage). To ensure diversity, patch geometries are varied within the 10 × 10 grid constraints.

Note: 10 specifically designed antennas for frequencies 1.5, 2, 2.4, 3, 3.5, 4, 4.6, 5.5, 6, 6.5 GHz are going to be simulated and included in 1000 data sets to improve the training data set quality.

3.3 Dataset Description

The training dataset is organized in an Excel file containing a total of 1,000 rows and 61 columns. Each row corresponds to one simulated antenna configuration, while the columns are divided into two groups: 100 input features representing the binary-encoded geometry of the patch antenna, and 61 output parameters representing the corresponding electromagnetic performance metrics S11 for each frequency points.

Table 1. Structure of the Training Dataset

Category	***Description***	***Quantity***
Input Features	Binary-encoded patch geometry (100 values)	100
Input Points	1x 100	
Output Parameters	Electromagnetic performance metrics: S11 (Frequency dependent)	1
Output Points	61 points from 1.0 to 6.0 GHz	122
Total Columns	Inputs + Outputs	222
Total Rows	Simulated antenna configurations	1,000
Training Dataset Excel	1000 x 161	

3.4 Model Architecture

A hybrid encoder-decoder model is employed:

- **Encoder:** A **Graph Neural Network (GNN)-Transformer** is used to model spatial dependencies in the antenna geometry. Each grid cell is treated as a node with edges representing adjacency, allowing the network to capture geometric interactions.
- **Decoder:** A **FEDformer (Fourier Enhanced Decomposition Transformer)** processes the latent representation to generate frequency-dependent output sequences (S-parameters and radiation metrics).

This hybrid architecture combines the spatial feature extraction capabilities of GNNs with the temporal/sequential forecasting strength of FEDformer.

3.5 Training Setup

- **Training Data Size:** 1000 simulation samples
- **Loss Function:** must satisfy the conditions below:

  (1) respects complex values, (2) cares more about resonant regions, and (3) reports application-level accuracy (resonant frequency, bandwidth, return loss).

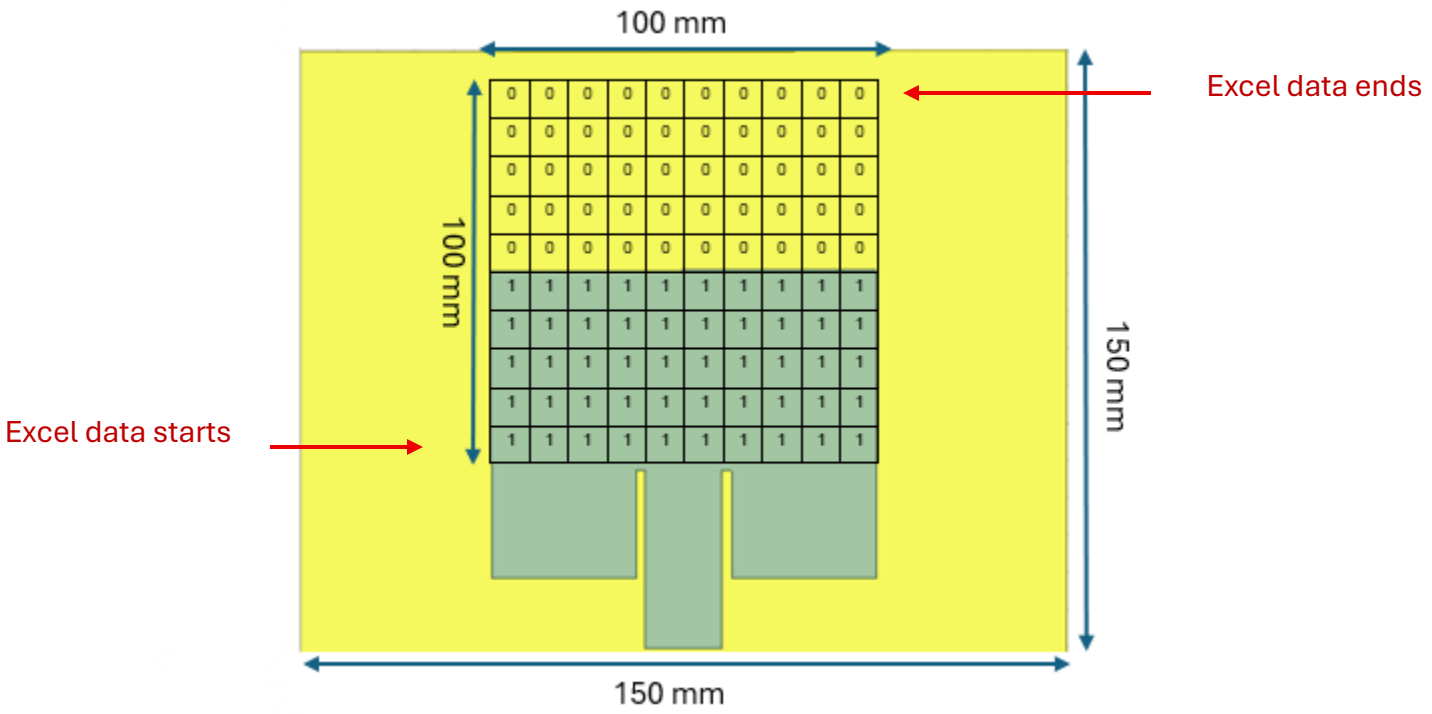
  - Complex Mean Squared Error (CMSE)

$$\frac{1}{N}\sum_{i=1}^{N}\frac{|S_i^{predicted} - S_i^{HFSS}|^2}{(S_i^{HFSS})^2}$$

- **Optimization:** Adam optimizer with learning rate scheduling

  (This is a widely used and often highly effective optimizer. It combines the benefits of Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp), adapting the learning rate for each parameter based on the first and second moments of the gradients. This makes it robust to sparse gradients and suitable for deep learning models like GNNs and FEDformer.)

- **Validation:** 20% of dataset reserved for testing



Excel Data Set

	Geometry									S11								
	Bottom left→Bottom right … Top left →Top right									1 GHz … →6 GHz								
First Simulation → 1000th Simulation	1	2	3	4	5	6	7	8	…	100	101	102	103	104	105	106	…	222
	0	1	1	0	1	0	0	0	…	1	-0.6	-0.8	-2.2	-4.5	-11	-5.2		-0.2
	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…

Research Team:

Electromagnetic research professor: Hao Xin hxin@arizona.edu

AI/ML research professor: Bo Liu boliu@arizona.edu

Students:

Hector Garcia hectorjgarcia@arizona.edu

Jude Coompson jcoompson@arizona.edu

Iman Miraki imanmiraki@arizona.edu

Appendix A. GPT vs Gemini Model Size Suggestion

Transformer	Topics	Chat GPT	Gemina
Encoder GNN	**Graph**	100 nodes K-NN edges K=8 (800 edges/Graph)	Since you have 100 geometry points, you can model them as nodes in a graph. The connections (edges) can be based on spatial proximity (e.g., k-nearest neighbors) or a predefined structure that reflects the antenna's layout. A fully connected graph is not advisable due to the computational cost and the lack of meaningful local structure.
	Type	GATv2	
	Message-passing layers	3	Start with **2 to 3 layers**.
	Hidden Dim	128	**64 to 128**
	Heads per Layer	4 (concat; keep output at 128)	
	Edge MLP	2 layers, 64 hidden	
	Node update MLP	2 layers, 128 hidden	
	Norm/Act	LayerNorm + GELU	
	Dropout	0.10	
	Readout	Global attention pooling	
	Geometry Embedding Size	256	
	Total encoder parameter	< 2-3 M for 1K samples	
Decoder	**D-model**	256	• **Number of Encoder/Decoder layers**: Start with **1 to 2 layers** for both the encoder and decoder. While FEDformer can handle long sequences efficiently, a large number of layers can lead to overfitting with only 1000 data samples. • **Attention Heads**: Use a low number of attention heads, such as **4 or 8**. More heads can add complexity and training time, which might not be beneficial for your dataset size. • **Hidden Dimension**: A hidden dimension of **256 to 512** is appropriate for the transformer blocks. This is a common practice for time-series transformers and provides sufficient capacity to learn the S11 curve's characteristics.
	N-heads	4	
	D-ff	512	
	E-layers	2	
	D-layers	1	
	Factor	6	
	Dropout	0.10 – 0.12	
	Position /frequency embedding	Learned 122 x D-model	
	Input Steam	frequency embeddings **tiled with** a single **[GEOM]** token = geom_emb (prepended)	
	Prediction length	122	

	Output head	linear → **2 channels** per f (ReΓ, ImΓ). (Compute |S11| dB for metrics.)	
Training	• **Optimizer: AdamW, LR 2e-3, weight decay 1e-4** • **Scheduler: cosine decay with 10% warmup** • **Batch size: 24–32** • **Epochs: 280–340, early stop (patience 30)** • **Grad clip: 1.0; mixed precision on** • **Split: 70/15/15 by geometry family (not random)**		
Regulation/Data Strategy	• **Geometry jitter ±1–2% (within manufacturable bounds); tiny feed-offset noise** • **Material jitter: εr ±0.05; tanδ small noise if available** • **Curriculum: train first on every other frequency (61 pts) for ~60–80 epochs, then fine-tune on all 122** • **Edge density check: avg degree ≈ 8–12**		

Chat GPT Suggestion

Got you, Iman. With **100 geometry nodes**, **122 frequency points**, and a **small dataset (~1,000 samples)**, keep the model compact and well-regularized. Here's a tuned, ready-to-train setup plus two variants.

Appendix A. Baseline (small-data tuned)

Graph (input)

- k-NN edges: **k=8–10** (≈800–1,000 edges/graph)
- Node feats: normalized (z-score). Include (x,y,z), widths, material flags, etc.
- Edge feats: Δx, Δy, distance, cosθ, sinθ (5–8 dims)

Encoder — GNN

- Type: **GATv2**
- Layers: **3**
- Hidden dim: **128**
- Heads/layer: **4** (concat; keep output at 128)
- Edge MLP: **2×64**
- Node update MLP: **2×128**
- Norm/act: **LayerNorm + GELU**
- Dropout: **0.10**
- Readout: **global attention pooling**
- Geometry embedding (**geom_emb**): **256**

Decoder — FEDformer (predict complex S11 over 122 f-points)

- d_model: **256**
- n_heads: **4**
- e_layers: **2**
- d_layers: **1**
- d_ff: **512**
- factor (Fourier sampling): **6**

- Dropout: **0.10–0.12**
- Frequency embeddings: **learned (122×d_model)**
- Conditioning: prepend a single **[GEOM]** token (the 256-d geom_emb) to the input sequence; the decoder cross-attends to it
- pred_len: **122**
- Output head: linear → **2 channels per f** (ReΓ, ImΓ)

Loss (notch-aware)

- Complex MSE over f + small **dB MAE** term (ε=1e-4)
- Weighting around minima: w(f)=1+2exp⁡(−((S11dB(f)−S11,min⁡dB)/4)2)w(f)=1+2\exp(-((S_{11}^{\text{dB}}(f)-S_{11,\min}^{\text{dB}})/4)^2)
- Optional scalar heads from geom_emb for $\mathbf{f_0}$ and **−10 dB BW** (MAE)
- Enforce |Γ|≤1|\Gamma|\le 1 via **tanh** on magnitude or penalty on max⁡(0,|Γ^|−1)\max(0,|\hat\Gamma|-1)

Training

- Optimizer: **AdamW**, LR **2e-3**, weight decay **1e-4**
- Scheduler: **cosine decay** with **10% warmup**
- Batch size: **24–32**
- Epochs: **280–340**, early stop (patience 30)
- Grad clip: **1.0**; mixed precision **on**
- Split: **70/15/15** by **geometry family** (not random)

Regularization / data strategy

- Geometry jitter ±1–2% (within manufacturable bounds); tiny feed-offset noise
- Material jitter: εr ±0.05; tanδ small noise if available
- **Curriculum:** train first on every **other** frequency (61 pts) for ~60–80 epochs, then fine-tune on all **122**
- Edge density check: avg degree ≈ 8–12

Baseline YAML

```
encoder_gnn:
  type: gatv2
  layers: 3
  hidden_dim: 128
  heads: 4
  edge_mlp_hidden: 64
  node_mlp_hidden: 128
  readout: attn_pool
  geom_emb: 256
  dropout: 0.10
  knn_k: 9

decoder_fedformer:
  d_model: 256
  n_heads: 4
  e_layers: 2
  d_layers: 1
  d_ff: 512
  factor: 6
  dropout: 0.12
  freq_points: 122
  conditioning: prepended_geom_token
  output: complex  # (Re, Im)

training:
  optimizer: adamw
```

lr: 0.002

weight_decay: 0.0001

batch_size: 28

epochs: 320

scheduler: cosine_warmup

warmup_frac: 0.10

grad_clip: 1.0

early_stop_patience: 30

Appendix A. If you see underfitting (val loss plateaus high)

"Plus" config (slightly larger)

- GNN: layers **4**, hidden **160**, heads **5** (keeps head_dim=32), dropout **0.12**, geom_emb **320**
- FEDformer: d_model **320**, n_heads **5**, e_layers **2–3**, d_layers **2**, d_ff **640**, factor **6**, dropout **0.12–0.15**
- Batch size **20–24**; LR **1.5e-3**

Appendix A. If you see overfitting (train≪val)

Lightweight config

- GNN: **GraphSAGE**, layers **3**, hidden **128**, dropout **0.10**, geom_emb **192**
- FEDformer: d_model **192**, n_heads **3**, e_layers **2**, d_layers **1**, d_ff **384**, factor **6**, dropout **0.10**
- Stronger aug; L2 **2e-4**; batch **32–40**; possibly label smoothing on |Γ|

Appendix A. What to monitor

- **MAE(|S11| dB)** overall + within **±50 MHz** of each predicted notch
- **Δf_0 (MHz)**, **ΔBW (MHz)**
- Violations of **|Γ|>1** (should be ≈0)

- Cross-correlation of |S11| curves to catch notch shifts

If you want, I can turn the baseline into a PyTorch (PyG + FEDformer) skeleton with the 100×122 shapes and the losses wired in.

Gemini Suggestion

A suitable model for this task would be an encoder-decoder architecture, with the **encoder GNN** to process the antenna geometry and the **decoder FEDformer** to predict the S11 curve. Given your dataset size of 1000, you'll need to select hyperparameters that balance model complexity with the risk of overfitting.

---

1. Encoder GNN (Geometry to Latent Representation)

The GNN's role is to convert the 100-point antenna geometry into a meaningful, fixed-size latent vector. This vector should capture the complex relationships between the geometric points.

- **Model Size**:
    - **Number of GNN layers**: Start with **2 to 3 layers**. Too few layers may not capture the deep geometric relationships, while too many could lead to oversmoothing, where node representations become too similar.
    - **Hidden Dimension**: A hidden dimension of **64 to 128** is a good starting point. This represents the size of the feature vector for each node in the graph. For your 100-point geometry, a hidden dimension in this range provides enough capacity without being overly complex.
    - **Graph Structure**: Since you have 100 geometry points, you can model them as nodes in a graph. The connections (edges) can be based on spatial proximity (e.g., k-nearest neighbors) or a predefined structure that reflects the antenna's layout. A fully connected graph is not advisable due to the computational cost and the lack of meaningful local structure.

2. Decoder FEDformer (Latent Representation to S11 Curve)

The FEDformer's role is to take the latent vector from the GNN and generate the 122-point S11 time series. FEDformer is particularly good at this due to its frequency-domain attention mechanism, which is efficient for long-sequence forecasting.

- **Model Size**:
    - **Number of Encoder/Decoder layers**: Start with **1 to 2 layers** for both the encoder and decoder. While FEDformer can handle long sequences efficiently, a large number of layers can lead to overfitting with only 1000 data samples.

- **Attention Heads**: Use a low number of attention heads, such as **4 or 8**. More heads can add complexity and training time, which might not be beneficial for your dataset size.
- **Hidden Dimension**: A hidden dimension of **256 to 512** is appropriate for the transformer blocks. This is a common practice for time-series transformers and provides sufficient capacity to learn the S11 curve's characteristics.

3. Hyperparameters for Training

These hyperparameters govern the learning process and are crucial for the model's performance.

- **Learning Rate**: A starting learning rate of **0.0001** is a safe choice. A lower learning rate prevents the model from diverging, especially with a small dataset.
- **Optimizer**: **Adam** or **AdamW** are robust and widely used optimizers that perform well in most deep learning tasks. AdamW is a good choice as it includes weight decay, which helps prevent overfitting.
- **Batch Size**: A **batch size of 16 or 32** is a good starting point. A smaller batch size can provide a more stable gradient estimate on a small dataset, but larger sizes can speed up training.
- **Epochs**: Train the model for around **100 to 200 epochs**. Use an early stopping mechanism to prevent overfitting. This involves monitoring a validation loss (e.g., on a 20% held-out set) and stopping training if it doesn't improve for a certain number of epochs.
- **Loss Function**: The **Mean Squared Error (MSE)** is a standard and effective loss function for this regression task, as it measures the average squared difference between the predicted and actual S11 values.
- **Dropout**: Apply **dropout** with a probability of **0.1 to 0.2** to both the GNN and FEDformer layers. Dropout helps regularize the model and prevents overfitting by randomly "dropping out" units during training.